



NEXUS ERP / DOCUMENTACION

Manual de Contabilidad

Como funciona cada submodulo del modulo contable: plan de cuentas, libros, cartera, estados financieros, conciliacion, activos fijos, cierres y la integracion automatica con el resto del ERP.

Contenido

Una sección por submodulo. Cada una explica que hace, como funciona, sus entradas y validaciones, y los asientos que genera.

1 Resumen contable

2 Plan de cuentas

3 Centros de costo

4 Impuestos y retenciones

5 Libros contables (Diario y Mayor)

6 Compras y gastos

7 Cuentas por pagar

8 Balance y estados financieros

9 Certificados de retención

10 Conciliación bancaria

11 Activos fijos y depreciación

12 Cierre contable (periodos)

13 Asientos recurrentes

14 Integración automática (el motor)

Resumen contable

Ruta: [/contabilidad](#) · Permiso: [accounting.view](#)

Es el tablero de entrada del contador. Reúne en una sola pantalla la foto financiera del momento, tomando los datos directamente del libro mayor (no de cifras digitadas a mano), de modo que siempre refleja la realidad de los asientos posteados.

Contexto del negocio: la empresa es una **petrolera que opera como agente retenedor** (compra bienes y servicios; no vende productos). Por eso el módulo está enfocado en compras, gastos, retenciones, activos y cuentas por pagar; el vertical de ventas/cartera por cobrar está oculto por no aplicar.

Qué muestra

- **Balance general** resumido (activo, pasivo, patrimonio).
- **Estado de resultados** del periodo (costos + gastos = resultado).
- **Flujo de efectivo** y posición de bancos.
- **Cuentas por pagar** y gastos del periodo, con accesos rápidos a compras y certificados.

De dónde salen los números

Todos los indicadores se calculan agregando las líneas de los asientos ([JournalLine](#)) respetando la naturaleza de cada cuenta (deudora/acreedora). No hay saldos "guardados a mano": si un asiento cambia, el resumen cambia.

Buena práctica: úsalo como primer chequeo diario. Si una cifra se ve rara, haz clic para bajar al detalle (libro mayor o auxiliar) y encontrar el asiento.

Plan de cuentas

Ruta: `/contabilidad/plan-de-cuentas` · API: `GET/POST/PATCH /accounting/accounts`

Es la columna vertebral de toda la contabilidad: el catálogo jerárquico de cuentas (Activo, Pasivo, Patrimonio, Ingreso, Costo, Gasto). Todo asiento se construye sobre estas cuentas.

Cómo está organizado

Estructura en árbol por código (1 dígito = clase, 2 = grupo, 4 = cuenta, etc.). Cada cuenta tiene:

Campo	Significado
<code>code</code>	Código único (ej. <code>1305</code>)
<code>type</code>	ASSET / LIABILITY / EQUITY / INCOME / COST / EXPENSE
<code>normalSide</code>	Naturaleza: DEBIT o CREDIT (define cómo suma)
<code>isPostable</code>	Solo las cuentas de movimiento (hojas) reciben asientos
<code>parentId</code>	Cuenta padre (para construir la jerarquía)

Regla de oro

Las cuentas **agrupadoras** (no postables) solo suman a sus hijas; **nunca** reciben movimientos directos. El sistema rechaza postear contra una cuenta no postable.

Caso real: la cuenta `15 Propiedad, planta y equipo` agrupa; el activo se registra en `1528 Equipo de cómputo` . La depreciación acumulada vive en `1592` , que es una cuenta de activo de **naturaleza crédito** (contra-activo): resta del activo. El sistema la trae configurada así desde el seed.

Centros de costo

Ruta: /contabilidad/centros-costo · **API:** GET/POST /accounting/cost-centers

Permiten clasificar ingresos y gastos por área, proyecto o unidad de negocio, para responder preguntas como "¿cuánto gastó Administración este mes?".

Cómo funciona

- Son **jerárquicos** (un centro puede tener un centro padre).
- Cada **línea** de un asiento puede llevar opcionalmente un **costCenterId**.
- Los reportes pueden filtrarse/agruparse por centro de costo.

Entradas y salidas

- **Entrada:** código + nombre + (opcional) centro padre.
- **Salida:** dimensión analítica disponible en todo el libro diario.

Recomendación: define pocos centros al inicio (Administración, Ventas, Producción) y haz **obligatorio** el centro de costo en las cuentas de gasto. Es la base de los informes de rentabilidad por área.

Impuestos y retenciones

Ruta: `/contabilidad/impuestos` · API: `GET/POST /accounting/tax-codes`

Motor de impuestos configurable: cada código define una tarifa, la cuenta contable donde se acumula y —para retenciones— la base sobre la que se calcula. Las tarifas se editan en datos, **sin tocar código**.

Estructura de un código

Campo	Significado
<code>code</code>	Identificador (ej. <code>IVA19P</code> , <code>RFTE_COMPRAS</code>)
<code>kind</code>	SALES / PURCHASE / WITHHOLDING (retención)
<code>rate</code>	Tarifa como fracción (<code>0.025</code> = 2.5%) — soporta hasta 6 decimales (ReteICA por mil)
<code>base</code>	Solo retenciones: SUBTOTAL (ReteFuente/ReteICA) o TAX (ReteIVA, sobre el IVA)
<code>accountId</code>	Cuenta GL donde se registra

Retenciones (la empresa como agente retenedor)

Al registrar una compra/gasto se eligen las retenciones a practicar. El sistema calcula cada una sobre su base y las **postea automáticamente** a su cuenta de pasivo, dejando la cuenta por pagar al proveedor por el **neto**:

Concepto	Cuenta PUC	Base
ReteFuente (renta)	<code>2365</code>	Subtotal
ReteIVA	<code>2367</code>	Sobre el IVA
ReteICA	<code>2368</code>	Subtotal

Vienen pre-cargados códigos típicos: compras 2.5%, servicios 4%, honorarios 11%, arrendamientos 3.5%, ReteIVA 15%, ReteICA 9.66×mil — todos editables.

Verificado de extremo a extremo: una compra de 1.000.000 + IVA 190.000 con ReteFuente 2.5% y ReteIVA 15% genera el asiento balanceado (Dr gasto + IVA / Cr retenciones 25.000 + 28.500 / Cr CxP neta 1.136.500). `prisma/smoke-withholding.ts`.

Libros contables — Diario y Mayor

Ruta: `/contabilidad/libros` · API: `/accounting/journal-entries` , `/accounting/ledger/:id`

Es el **corazón** del módulo. Todo —ventas, compras, depreciación, cierres— termina siendo un asiento aquí.

Libro Diario (asientos)

Lista cronológica de asientos. Cada asiento (`JournalEntry`) tiene varias líneas y cumple **partida doble**: la suma de débitos = suma de créditos.

`JournalService.createEntry` es el **único punto de posteo** del sistema y garantiza:

1. **Cuadre**: rechaza el asiento si débitos $\neq$ créditos.
2. **Una línea = débito O crédito** (nunca ambos, nunca cero).
3. **Periodo abierto**: bloquea asientos en periodos cerrados/bloqueados.
4. **Numeración correlativa** vía contador atómico (`Sequence`).
5. **Idempotencia**: un documento origen un solo asiento (`sourceType` + `sourceId`).

Inmutabilidad

Un asiento posteoado **no se edita ni se borra**. Para corregir se genera una **reversión** (asiento espejo que invierte débitos y créditos) y ambos quedan enlazados. Esto es obligatorio para auditoría.

Libro Mayor (por cuenta)

Para una cuenta y un rango de fechas muestra: saldo inicial, cada movimiento, saldo corriente y totales — respetando la naturaleza de la cuenta.

Diagrama: Documento origen evento `createEntry` (valida) `JournalEntry` + `JournalLine[]`
suma al Mayor alimenta reportes.

Compras y gastos

Ruta: `/contabilidad/compras` · API: `/purchases/bills` , `/parties`

Es la operación diaria de la petrolera: registrar facturas de proveedores y contratistas, aplicarles las retenciones y pagarlas. Todo genera su asiento solo.

Registrar una factura

1. Eliges el **proveedor** (o lo creas en la pestaña Proveedores).
2. Capturas subtotal, IVA descontable y la **naturaleza** (gasto/servicio o inventario).
3. Marcas las **retenciones** a practicar; se muestra una **vista previa en vivo** del asiento (subtotal IVA total bruto retenciones **neto a pagar**).
4. Al guardar, emite `purchase.bill.created` y se postea automáticamente con la cuenta por pagar **neta** de retenciones.

Pagar una factura

Desde la lista, el botón **Pagar** registra el egreso: elige la cuenta bancaria y el monto (precargado con el saldo). Esto registra la salida de banco y emite `payment.made` asiento `Dr CxP / Cr Banco` , bajando el saldo del proveedor.

Verificado de extremo a extremo: factura neta 1.136.500 pago parcial 500.000 (queda PARCIAL) pago final (queda PAGADA); cada pago genera su asiento balanceado.

Cuentas por pagar

Ruta: [/contabilidad/cartera](#) · API: [/accounting/payables](#) (+ [/aging](#))

Muestra lo que la empresa debe a proveedores y contratistas, **derivado de las facturas de compra** (sin duplicar información), con análisis de antigüedad para programar pagos.

Cómo funciona

- Se alimenta de las **facturas de proveedor** pendientes (estado ISSUED/PARTIAL).
- **Aging** en cubetas: corriente, 1–30, 31–60, 61–90 y +90 días.
- El saldo refleja el **neto** (después de retenciones practicadas).

Asientos relacionados (automáticos)

Evento	Asiento
Factura de compra	Dr Gasto Inventario + Dr IVA / Cr Retenciones / Cr CxP neta
Pago a proveedor	Dr CxP / Cr Banco

Indicador clave: el DPO (días de pago) sale de aquí y mide el ciclo de efectivo. *(El vertical de cuentas por cobrar existe en el backend pero está oculto: la petrolera no vende.)*

Balance y estados financieros

Ruta: </contabilidad/informes> · API: /accounting/reports/*

Cuatro reportes oficiales generados en vivo desde el libro mayor:

Reporte	Endpoint	Qué responde
Balance de comprobación	/reports/trial-balance	¿Cuadra la contabilidad? ($\Sigma \text{ Dr} = \Sigma \text{ Cr}$)
Balance general	/reports/balance-sheet	¿Qué tengo, qué debo, mi patrimonio?
Estado de resultados	/reports/income-statement	¿Gané o perdí en el periodo?
Flujo de efectivo	/reports/cash-flow	¿Cómo se movió el efectivo?

Cómo se calculan

Agregan las líneas de asientos POSTED/REVERSED respetando la naturaleza de cada cuenta y el rango de fechas. El **balance de comprobación** es la validación maestra: si no cuadra, hay un problema de datos que revisar antes de cerrar.

Exportación

Cada estado se puede **exportar a PDF** (vista imprimible) y a **Excel** ([.xls](#) que abre nativo, con números como celdas reales). Botones disponibles en cada pestaña del informe.

Certificados de retención

Ruta: `/contabilidad/certificados` · API: `/accounting/reports/withholding-*`

Genera los certificados que la empresa, como agente retenedor, **debe emitir a sus proveedores** por las retenciones practicadas en el año gravable (ReteFuente, ReteIVA, ReteICA).

Cómo funciona

1. Eliges el **año gravable**; se listan los proveedores con retenciones y su total.
2. Cada proveedor genera su **certificado imprimible / PDF** en una ventana lista para guardar.
3. Los datos se **reconstruyen desde los asientos**: enlaza cada retención al proveedor vía la factura de origen y agrupa por cuenta (`2365` / `2367` / `2368`) — que corresponden a los tres certificados legales (Renta, IVA, ICA).

Datos del agente retenedor

La razón social y el NIT que encabezan el certificado se configuran por variables de entorno:

`COMPANY_NAME` , `COMPANY_NIT` , `COMPANY_ADDRESS` , `COMPANY_CITY` .

Verificado: dos facturas de un proveedor con ReteFuente 2.5% y ReteIVA agregan correctamente el total retenido del año por concepto.

Conciliación bancaria

Ruta: `/contabilidad/conciliacion` · API: `/accounting/reconciliation/*`

Compara lo que dice el banco (extracto) contra lo que dice tu contabilidad (movimientos del libro), para detectar diferencias.

Cómo funciona

1. **Vista lado a lado:** movimientos contables vs. líneas del extracto.
2. **Conciliación automática:** empareja por monto (`auto-match`).
3. **Conciliación manual:** marcas movimiento línea de extracto; valida que los montos coincidan.
4. Lo que no concilia queda como **partida pendiente** (cheques no cobrados, comisiones, GMF) para investigar.

Entradas y validaciones

- **Entrada:** líneas de extracto (`BankStatementLine`) + movimientos (`BankMovement`).
- **Validación:** un movimiento no se concilia dos veces; montos deben coincidir.

Pendiente identificado: importación de extracto desde archivo (CSV/OFX). Hoy las líneas del extracto deben existir en la tabla; el emparejado ya funciona.

Activos fijos y depreciación

Ruta: </contabilidad/activos-fijos> · API: /accounting/fixed-assets/*

Módulo **nuevo** que gestiona la propiedad, planta y equipo: alta, depreciación mensual automática y baja/venta, **generando todos los asientos por sí solo**.

Categorías

Cada activo pertenece a una categoría que define la **política de depreciación** y las tres cuentas contables que usará:

Cuenta	Ejemplo
Activo (costo)	1528 Equipo de cómputo
Gasto depreciación	5160 Depreciaciones
Depreciación acumulada	1592 (contra-activo)

Vienen pre-cargadas: **COMP** (cómputo, 36 meses), **MUEB** (muebles, 120), **VEHIC** (vehículos, 60).

Ciclo de vida y asientos

Paso	Asiento automático
Alta (capitalización opcional)	Dr Activo / Cr Banco o CxP
Depreciación mensual	Dr Gasto depreciación / Cr Depreciación acumulada
Baja / venta	Dr Dep. acumulada + Dr Banco (venta) / Cr Activo + Cr/Dr utilidad o pérdida

Depreciación (línea recta)

Cuota mensual = (costo – valor residual) ÷ vida útil en meses. El sistema:

- **Nunca** deprecia por debajo del valor residual.
- Es **idempotente**: un activo se deprecia **una sola vez** por mes (control **(activo, año, mes)**), así que re-ejecutar no duplica.
- Marca el activo como *Totalmente depreciado* al llegar al valor residual.

Baja con utilidad/pérdida

Al dar de baja, calcula valor en libros (costo – acumulada), lo compara con el valor de venta y registra automáticamente la **utilidad** (4248) o **pérdida** (5310).

Verificado: prueba de extremo a extremo — activo de 3.600.000 a 36 meses deprecia 100.000/mes, acumula correctamente, todos los asientos quedan balanceados, y una venta por 3.500.000 sobre valor en libros de 3.400.000 registra utilidad de 100.000. (`prisma/smoke-fixed-assets.ts`).

Cierre contable — periodos

Ruta: [/contabilidad/cierre](#) · API: [/accounting/periods/*](#)

Controla qué meses están abiertos para registrar y ejecuta los cierres.

Estados de un periodo

Estado	Significado
OPEN	Admite nuevos asientos
CLOSED	Cerrado; no admite asientos (reabrible)
LOCKED	Bloqueado definitivamente

Operaciones

- **Abrir mes:** crea el periodo (valida mes 1–12, fija fechas inicio/fin).
- **Cerrar / Reabrir:** controla el registro mes a mes.
- **Cierre anual ([close-year](#)):** cancela las cuentas de resultado (ingresos, costos, gastos) contra **resultados acumulados**, dejándolas en cero para el nuevo año.

Por qué importa

Cerrar un periodo **protege la historia**: impide que alguien modifique meses ya reportados a gerencia o a la DIAN. El motor de asientos respeta este bloqueo.

Flujo mensual sugerido: conciliar bancos correr depreciación provisiones liquidar impuestos revisar balance de comprobación (debe cuadrar) **cerrar mes**.

Asientos recurrentes

Ruta: integrado en contabilidad · **API:** `/accounting/recurring/*`

Automatiza asientos que se repiten cada mes (arrendos, amortizaciones, provisiones fijas) para que el contador no los teclee una y otra vez.

Cómo funciona

1. Defines una **plantilla** con sus líneas (cuentas, débitos/créditos) y una **frecuencia** (mensual, trimestral, anual) + fecha de próxima ejecución.
2. Al llegar la fecha, el sistema **materializa** la plantilla en un asiento real y avanza la fecha al siguiente periodo.
3. Es **idempotente**: no duplica si se corre dos veces.

Automatización

Un proceso programado (cron) revisa **a diario** las plantillas vencidas y las ejecuta sin intervención. También puede dispararse manualmente desde la API (`/recurring/run`).

Beneficio: elimina trabajo repetitivo y errores de digitación en los devengos fijos del cierre mensual.

Integración automática — el motor de asientos

Componentes: `PostingService`, `PostingListeners`, `AccountMapping`, eventos de dominio.

Es lo que convierte a contabilidad en parte de un **ERP** y no en un programa aislado: las demás áreas (Ventas, Compras, Inventario, Bancos) **no saben de contabilidad**; solo emiten eventos. Contabilidad los escucha y postea.

Eventos que escucha

Evento	Asiento que genera
<code>purchase.bill.created</code>	Dr Gasto Inventario + Dr IVA / Cr Retenciones / Cr CxP neta
<code>payment.made</code>	Dr CxP / Cr Banco
<code>inventory.cost.posted</code>	Dr Costo / Cr Inventario

(El backend también soporta `sales.invoice.created` y `payment.received`, pero no se usan en esta operación porque la petrolera no vende.)

El mapeo de cuentas (clave del diseño)

Las cuentas **no están escritas en el código**. Viven en la tabla `AccountMapping` (`SALES_AR`, `SALES_REVENUE`, `COGS`, `RETAINED_EARNINGS`, etc.). Si quieres que las ventas peguen a otra cuenta, cambias el mapeo en datos — sin tocar programación.

Robustez

- **Idempotencia** por `(sourceType, sourceId)`: reintentos no duplican asientos.
- Los errores de posteo se **registran** sin romper la operación de la otra área.

Por qué es valioso: el contador deja de digitar el 80% de los asientos. Su trabajo pasa de *capturar a revisar y conciliar* — que es donde aporta valor.